# Modular Design, Testbenches & Inferring Hardware

Slides by Viorela Ila and Lewis Watts

# Modular Design in Verilog

Given the principal design entity in Verilog is a module, one could infer that Verilog designs should be modular. Many tools are provided in Verilog to make this happen, including:

- Component Instantiation
- Looping
- Generate Blocks
- Tasks and Functions

# Component Instantiation in Verilog

- Use of component or module instantiations is the fundamental way to build hierarchy in a design.

- A **module** provides a template from which you can create actual objects. When a module is invoked, Verilog creates a unique object from the template. Each object has its own name, variables, parameters, and I/O interface.

- The process of creating objects from a module template is called **instantiation**, and objects are called **instances**.

# Component Instantiation in Verilog

```
//  4-bit Adder
module add4  (
  input [3:0] Data1,
  input [3:0] Data2,
  input Cin,
  output Cout,
  output [3:0] Sum
);
   assign {Cout, Sum} = Data1 + Data2 + Cin;
endmodule
```

How to implement a **16-bit** adder reusing the implementation of the **4-bit** adder?

# Component Instantiation in Verilog

```
//  4-bit Adder
module add4  (
  input [3:0] Data1,
  input [3:0] Data2,
  input Cin,
  output Cout,
  output [3:0] Sum
);
   assign {Cout, Sum} = Data1 + Data2 + Cin;
endmodule
```

```
//  16-bit Adder: 4 instantiations of 4-bit Adders
module Add16bit  (
  input [15:0] A,
  input [15:0] B,
  input Cin,
  output Cout,
  output [15:0] Sum
);
  wire Cin2, Cin3, Cin4; // intermediate Cin
  add4 add4_1 (.Data1(A[3:0]), .Data2(B[3:0]), .Cin(Cin), .Cout(Cin2), .Sum(Sum[3:0]));
  add4 add4_2 (.Data1(A[7:4]), .Data2(B[7:4]), .Cin(Cin2), .Cout(Cin3), .Sum(Sum[7:4]));
  add4 add4_3 (.Data1(A[11:8]), .Data2(B[11:8]), .Cin(Cin3), .Cout(Cin4), .Sum(Sum[11:8]));
  add4 add4_4 (.Data1(A[15:12]), .Data2(B[15:12]), .Cin(Cin4), .Cout(Cout), .Sum(Sum[15:12]));
endmodule
```

**port name!**

# Component Instantiation in Verilog

```
//  4-bit Adder
module add4  (
  input [3:0] Data1,
  input [3:0] Data2,
  input Cin,
  output Cout,
  output [3:0] Sum
);
   assign {Cout, Sum} = Data1 + Data2 + Cin;
endmodule
```

```
//  16-bit Adder: 4 instantiations of 4-bit Adders
module Add16bit  (
  input [15:0] A,
  input [15:0] B,
  input Cin,
  output Cout,
  output [15:0] Sum
);
  wire Cin2, Cin3, Cin4; // intermediate Cin
  add4 add4_1 (A[3:0], B[3:0], Cin, Cin2, Sum[3:0]);
  add4 add4_2 (A[7:4], B[7:4], Cin2, Cin3, Sum[7:4]);
  add4 add4_3 (A[11:8], B[11:8], Cin3, Cin4, Sum[11:8]);
  add4 add4_4 (A[15:12], B[15:12], Cin4, Cout, Sum[15:12]);
endmodule
```

**ordered port lists!**

# Instantiation with Parameters

Syntax:

```
module module_name_top #(parameter list) (port connection list);
  module_name_1 #(parameter list) instance_name_1 (port connection list);
  module_name_2 #(parameter list) instance_name_2 (port connection list);
  .......
  module_name_n #(parameter list) instance_name_n (port connection list);
endmodule
```

Parameter List:

```
#(parameter param_name_1 = default_val_1, ..., param_name_n = default_val_n)
```

Port Connection List:

```
(input input_1, input_2, output output_1, output reg [3:0] output_2, input [2:0] input_3)
```

# Instantiation with Parameters

Changed adder to N-bit using the parameter N →

```
//  N-bit Adder
module addN #(parameter N)  (
  input [N-1:0] Data1,
  input [N-1:0] Data2,
  input Cin,
  output Cout,
  output [N-1:0] Sum
);
   assign {Cout, Sum} = Data1 + Data2 + Cin;
endmodule
```

```
//  16-bit Adder: instantiation of an N-bit Adder with N=16
module Add16bit  (
  input [15:0] A,
  input [15:0] B,
  input Cin,
  output Cout,
  output [15:0] Sum
);

  addN #(.N(16)) add16 (.Data1(A), .Data2(B), .Cin(Cin), .Cout(Cin2), .Sum(Sum));

endmodule
```

# Recap: Blocking Assignments

- Procedural (blocking) assignments (=) are done sequentially in the order the statements are written. A second assignment is not started until the preceding one is complete.
  - *Use for combinational logic.*
- Procedural (nonblocking) assignments (<=), which follow each other in the code, are evaluated in parallel.
  - *Use for sequential logic.*

**Blocking: Order matters!**

```
always @( posedge clk)
    begin
        Q2=Q1; Q1=D; // is a bad shift register
    end
```

**Vs.**

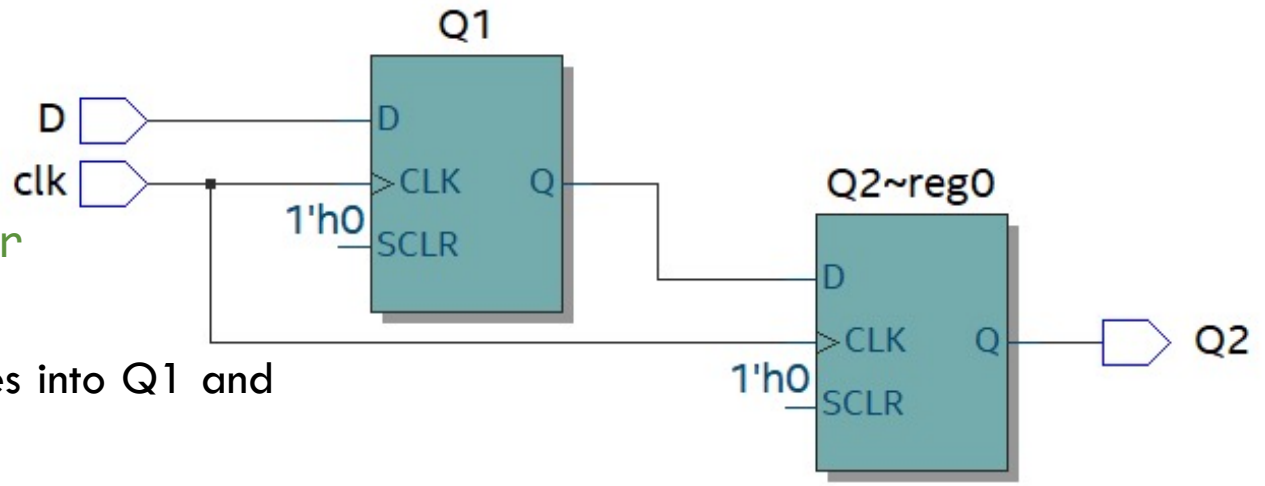
```
always @( posedge clk)
    begin
        Q1=D; Q2=Q1; // is a single or parallel flip-flop
    end
```

# Recap: Blocking Assignments

**Blocking: Order matters!**

```
always @( posedge clk)
    begin
        Q2=Q1; Q1=D; // is a bad shift register
    end
```

**Quartus Synthesizer Output:**

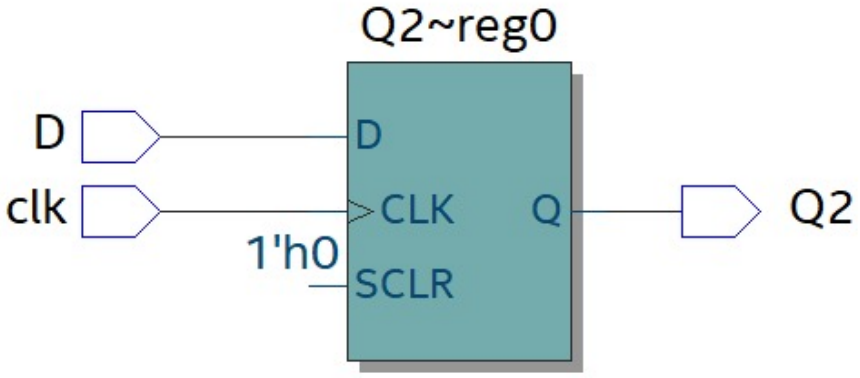



The synthesizer creates a **shift register** where you have D goes into Q1 and then Q1 goes into Q2 and so on.

# Vs.

The synthesizer is going to either generate a **single flip-flop** where D is the input and Q2 is the output, or it might create **parallel flip-flops** where you have both Q1 and Q2 outputs, but D is driving both of those.

```
always @( posedge clk)
    begin
        Q1=D; Q2=Q1; // is a single or parallel flip-flop
    end
```



# Recap: Nonblocking Assignments

- Procedural (nonblocking) assignments (<=), which follow each other in the code, are evaluated in parallel.
  - *Use for sequential logic!*

When you have **nonblocking** assignments and they follow each other in the code, they're all evaluated in parallel, so they are concurrent.

**Non-Blocking: Order does not matter!**

```
always @( posedge clk)
    begin
        Q2<=Q1; Q1<=D; // shift register
    end
always @( posedge clk)
    begin
        Q1<=D; Q2<=Q1; // shift register
    end
always @( posedge clk)
    begin
        Q1<=D; Q2<=D;  // parallel flip-flop
    end
```

# Looping in Verilog

There are several looping constructs within Verilog, including:

- **repeat**
- **while**
- **forever**
- **for**

The **for** loop statement is written and behaves just like it does in C, as is the **while** loop.

The **forever** loop is explicitly an infinite loop.

The **repeat** statement executes a statement or block of statements a fixed number of times.

# For - loop

- A **for** loop is used to test all combinations or to replicate hardware in Verilog.
- It iterates a set of statements given within the loop as long as the given condition is true.
- Is used in the context where an **iterator** is available and the condition depends on the value of the iterator.

```
for (<initial_condition>; <condition>; <step_assignment>) begin
   // Statements
end
```

# Looping in Verilog – Example

- Example: assigning each bit of a vector.
- Can you think of a more concise way of doing this without a loop?

```
assign tmp = a & {(N){g}};
```

```
//  AND scalar g with vector A
module scalarAnd #(parameter N = 4)
(  input g,
   input [N-1:0] a,
   output [N-1:0] y
);

  reg [N-1:0] tmp, y;
  integer i;   // loop index, not signal!

  always @(a or g)
  begin
    for(i=0; i<N; i=i+1)
      begin : ands // Quartus: Must give for loop a label.
        tmp[i] = a[i] & g;
      end
    y = tmp;
  end

endmodule
```

← Module parameters

← Loop for each bit in tmp and a

# Looping in Verilog – Better Example

- Example:

| Input data_in | encoded_out | valid | Explanation |
|---|---|---|---|
| 8'b10000000 | 3'b111 (7) | 1'b1 | Highest priority bit is at position 7 |
| 8'b01000000 | 3'b110 (6) | 1'b1 | Highest priority bit is at position 6 |
| 8'b01010000 | 3'b110 (6) | 1'b1 | Both bits 6 and 4 are set, but 6 has higher priority |

```
//  Priority Encoder: Finds the position of the highest
priority (leftmost) '1' bit
module priority Encoder: #(parameter N = 8)
(  input  [N-1:0] data_in,
   output reg [$clog2(N)-1:0] encoder_out,
   output reg valid
);

  integer i;   // loop index, not signal!
  always @(*) begin
    encoder_out = 0;
    valid = 0;
// Scan LSB to MSB; higher bits overwrite lower bits
for(i = 0; i < N; i = i+1) begin : encode
        if (data_in[i] == 1'b1) begin
                encoder_out = i;
                valid = 1’b1;
        end
    end
  end
endmodule
```

# While - loop

- The **while** loop is used to run some code for an indeterminate amount of time.
- A **while** loop does some action until the condition it is checking is no longer true.
- While loops are a part of Verilog, however **it is not recommend using while loops for synthesizable code!!**

- While loops can be very useful in your **testbenches**! When some code needs to run an indeterminate amount of loops, a **while** loop can do the job.

```
while (<condition>) begin
   // Multiple Statements
end
```

```
do begin
   // Multiple Statements
end while (<condition>)
```

# While - loop

The code waits for a ready signal to go high, counts clock cycles while waiting, displays the result when condition is met.

```
// Simple testbench with while loop
module tb;
    reg clk = 0, ready = 0;
    integer count = 0;
    // Clock generation
    always #5 clk = ~clk;

    // Make ready go high after some time
    initial #50 ready = 1;

    // Test using while loop
    initial begin // Wait until ready signal is high
        while (!ready) begin
            @(posedge clk);
            count = count + 1;
        end
        $display("Ready after %0d clock cycles", count);
        $finish;
    end
endmodule
```

# Repeat

The `repeat` construct in Verilog is used to execute a block of statements a specified number of times. It is primarily used within procedural blocks like always, initial, or in tasks and functions.

```
repeat (expression) begin
    // Block of statements
end
```

```
module counter_example;
    reg [3:0] counter;
    reg clk;

    always @(posedge clk) begin
        counter = 0;
        repeat (8) begin
            counter = counter + 1;
        end
    end
endmodule
```

# Generate in Verilog

Although loops can be used to generate data or test patterns, in Verilog a common use of loops for synthesis is replication of many identical circuits (modules) within **generate** blocks.

- The **generate** specifies how an object is to be repeated.
- Variables used to specify the repetition are called **genvar**s and they tell the tool that this variable is to be specifically used during the elaboration of the **generate** block.
- The index variable of a for loop in a generate block must be a **genvar**.
- One can use **generate if** construct to select between different implementations.

# Generate in Verilog : Looping Continuous Assignments

Example of a generate block →

Generate for loop →

```
//  Generate n XOR gates
// In this example, we XOR two vectors one bit at a time.

module XorGen #(parameter width = 4, delay = 10)
(  output [width-1:0] xout,
   input [width-1:0] xin1, xin2
);

 genvar i;
 generate
        for (i =0; i < width; i=i+1)
        begin : xors // Quartus: Must give for loop a label.
               assign #delay xout[i] = xin1[i] ^ xin2[i];
        end
  endgenerate

endmodule
```

# Generate in Verilog : Looping Module Instantiations

```
//  16-bit Adder: 4 instantiations of 4-bit Adders using a generate-for loop
module Add16bit  (
  input [15:0] A,
  input [15:0] B,
  input Cin,
  output Cout,
  output [15:0] Sum
);

  wire [4:0] C; // intermediate Carry signals using a vector
  generate
    genvar i;
    for (i=0; i<4; i=i+1) begin : adders    // Quartus: Must give for-loop a label.
      add4 u (.Data1(A[(i+1)*4-1 : i*4]), .Data2(B[(i+1)*4-1 : i*4]), .Cin(C[i]),
                .Cout(C[i+1]), .Sum(Sum[(i+1)*4-1 : i*4]) );
    end
  endgenerate
  assign C[0] = Cin;
  assign Cout = C[4];
endmodule
```

```
add4 add4_1 (.Data1(A[3:0]),  .Data2(B[3:0]),  .Cin(C[0]), .Cout(C[1]), .Sum(Sum[3:0]));
add4 add4_2 (.Data1(A[7:4]),  .Data2(B[7:4]),  .Cin(C[1]), .Cout(C[2]), .Sum(Sum[7:4]));
add4 add4_3 (.Data1(A[11:8]), .Data2(B[11:8]), .Cin(C[2]), .Cout(C[3]), .Sum(Sum[11:8]));
add4 add4_4 (.Data1(A[15:12]),.Data2(B[15:12]),.Cin(C[3]), .Cout(C[4]), .Sum(Sum[15:12]));
```

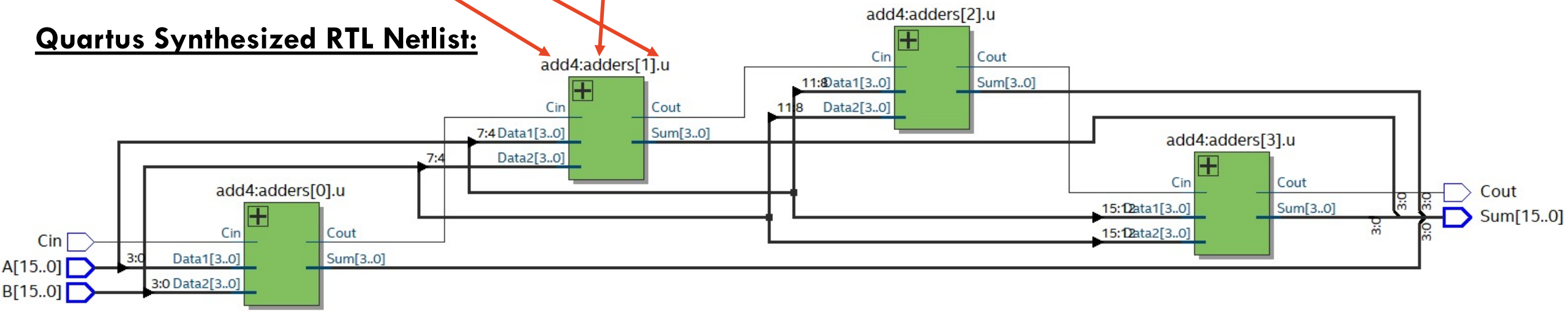
← Looped module instantiation

# Generate in Verilog : Looping Module Instantiations

```
wire [4:0] C; // intermediate Carry signals using a vector
generate
  genvar i;
  for (i=0; i<4; i=i+1) begin : adders // Quartus: Must give for-loop a label.
    add4 u (.Data1(A[(i+1)*4-1 : i*4]), .Data2(B[(i+1)*4-1 : i*4]), .Cin(C[i]),
            .Cout(C[i+1]), .Sum(Sum[(i+1)*4-1 : i*4]) );
  end
endgenerate
```

**Module name, for-loop name and instance name.**

**Quartus Synthesized RTL Netlist:**



# Generate in Verilog : If Statements

```
module parametric_buffer #(parameter INVERT = 0) (
    input  in,
    output out
);
    generate
        if (INVERT)
            assign out = ~in;
        else
            assign out = in;
    endgenerate

endmodule
```

← Generate if statement based on parameter value

- Generate blocks are a bit like constant expressions in C++ in that they must be evaluated at “compile-time”.
- So, you can only use constants in the if conditions, like parameters and localparams.

# Tasks and Functions

- Verilog code can be further modularised using *Tasks* and *Functions*. These **can be called in procedural blocks.**
- Allow breaking up large procedural blocks (e.g. **always**, **initial**) into smaller ones and encourage **code re-use.**
- They are synthesizable as long as the code inside them is synthesizable.
- They can belong to modules, or outside of modules (a global task/function).
- Both are static by default (variables are shared) – use the **automatic** keyword to ensure each invocation dynamically allocates its own variables.

**Syntax Form 1**

```
task automatic sum;
input  [15:0] A, B;
output [15:0] C;
begin
   C = A + B;
end
endtask
// Call using sum(a, b, c);
```

```
function automatic [15:0] sum;
input  [15:0] A, B;
begin
  sum = A + B;
end
endfunction

// Call using sum(a, b);
```

# Tasks Versus Functions

**Tasks**

- can have zero or more arguments.
- have as many **output**, **input** or **inout** type arguments as necessary.
- do not return values; rather, they have output arguments, similar to how a module has output ports.
- can take multiple simulation time units.

**Functions**

- must execute in the same simulation time unit, so they ***cannot*** have delay statements, etc.
- must have at least one **input** argument. Functions cannot have **output** or **inout** arguments.
- only have one output, which is returned and is implicitly defined as the function name.

**Syntax Form 2**

```
task sum (input [15:0] A, B, output [15:0] C);
 begin
   C = A + B;
 end
endtask
// Call using sum(a, b, c);
```

```
function [15:0] sum (input  [15:0] A, B);
 begin
   sum = A + B;
 end
endfunction
// Call using sum(a, b);
```

# HDL Simulator

## Simulation ≠ Synthesis

| Simulation Only | Synthesis Only | Both |
|---|---|---|
| Delay statements (#10) | 'X' propagation handling | Basic logic operations |
| File I/O operations | Some pragma directives | Registers and wires |
| Display tasks ($display) | Clock domain assumptions | Control structures |

**What is an HDL Simulator?**

**An HDL Simulator is just a Software Program that:**

- Models hardware behavior using software
- Creates the *illusion* of hardware concurrency
- Includes programming constructs and interactive elements

**The Golden Rule**

**Your HDL code should both simulate correctly AND be synthesizable**

**Verilog Language Subsets**



# Testbenches in Verilog

- A testbench is a program written in any language (can be Verilog) for the purposes of **exercising and verifying** the **functional correctness** of the hardware model as coded.
- The use of the testbench to auto-generate a **stimulus** for the **unit under test** is much more efficient than entering test vectors manually in the simulators like *Modelsim*.
- Also known as a **test fixture** or **test harness.**
- A  powerful tool for auto-generating test stimulus (e.g. test signals) and test results.
- Most designers spend at least as much time creating testbench code as they do the code for the unit under test.

# Testbenches in Verilog

A **testbench** can have several functional sections, including:

a) Top-level testbench *declaration*

b) **Stimulus** and **Response Signal** *declarations*

c) Component *declarations*

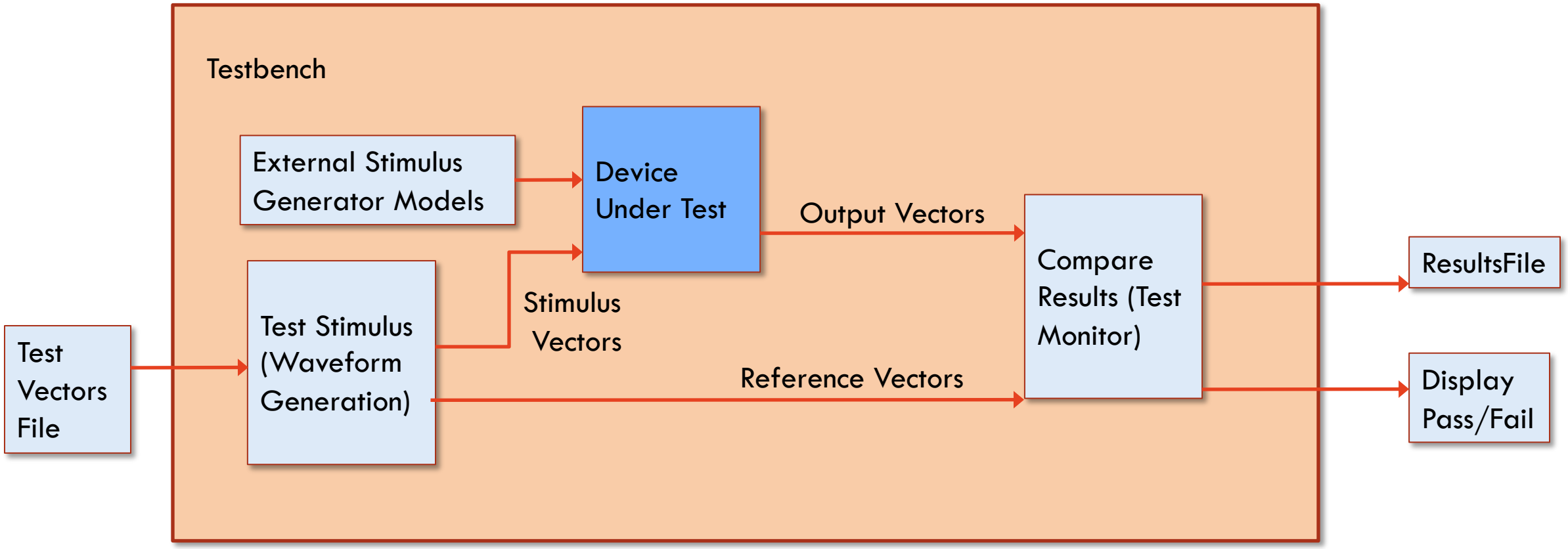
d) Component **device under test** (DUT) instantiations

e) External stimulus generator

f) Test process which applies the stimulus to the DUT

g) Test **monitor** which reports results

# Testbench Structure



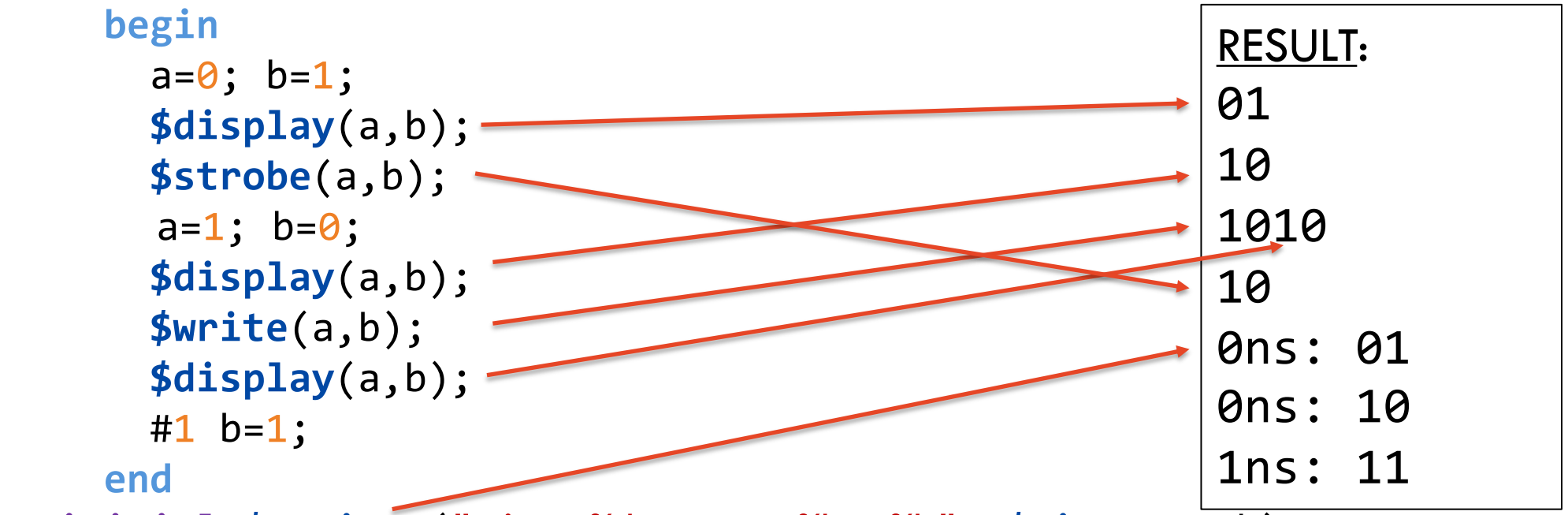
# Display/Write, Monitor/Strobe …

Although all **$display, $monitor, $write** and **$strobe** in Verilog seem to be similar, there is a slight difference:

- **$display** is the normal display which executes on its parameters wherever it is present in the code.
- **$write** is similar to **display** except that **display** displays the contents in the next line (cursor moves to the next line before displaying) unlike **write** in which the contents are displayed in the same line.
- **$strobe** executes only once in a time instant when all processes in that time instant have executed.
- **$monitor** executes only if any of its parameters change (in one time instant)

# Display/Write, Monitor/Strobe …

```
module d11;
  reg a=0,b=0;
  initial
    begin
      a=0; b=1;
      $display(a,b);
      $strobe(a,b);
       a=1; b=0;
      $display(a,b);
      $write(a,b);
      $display(a,b);
      #1 b=1;
    end
  initial $monitor("time=%d, ns:, %b, %b", $time, a, b);
endmodule
```




**$strobe(a,b);**

•Displays the values of a and b at the end of the current simulation time (after all other events scheduled at this time are processed).

# Adder Test Bench

Compiler directive that specifies the time unit and precision

```
`timescale <time_unit>/<time_precision>

// Example
`timescale 1ns/1ps
`timescale 10us/100ns
`timescale 10ns/1ns
```

```
`timescale 1 ns / 1 ps  // set timescale to
                        // nanoseconds, ps precision
module Adder_tb;        //  no port list!
  // Step 1: Signal Declarations
  reg [3:0] a_tb, b_tb;  // data input stimulus
  reg Cin;               // data input stimulus
  wire [3:0] y_tb;       // data output response
  wire Co_tb;            // data output response
  reg [3:0] expected;    // expected sum result

  // Step 2: Device Under Test (DUT) instantiation
  add4 DUT(.Data1(a_tb), .Data2(b_tb), .Cin(Cin), .Sum(y_tb), .Cout(Co_tb));

  // To be continued ...
```

# Adder Test Bench

```
  // Adder Testbench, continued
// Step 3: Test Stimulus Generation Variant 0
  initial
    begin
          a_tb=2;  b_tb=2; Cin=0; expected=4;
      #10 a_tb=15; b_tb=0; Cin=1; expected=0; // # is the delay operator.
      #10 a_tb=2;  b_tb=4; Cin=1; expected=7; // Delay 10 time units, then change values.
      #10 $stop;
    end
  // Step 4: Test Results
  initial
    $monitor("time=%d, a=%b, b=%b, Cin=%b, sum=%b, cout=%b, expected sum=%b",
                  $time, a_tb, b_tb, Cin, y_tb, Co_tb, expected);
endmodule
```

```
time=0, a=0010, b=0010, Cin=0, sum=0100, cout=0, expected sum=0100
time=10, a=1111, b=0000, Cin=1, sum=0000, cout=1, expected sum=0000
time=20, a=0010, b=0100, Cin=1, sum=0111, cout=0, expected sum=0111
```

# Adder Test Bench with Loops

```
integer i, j, k;
// Adder Testbench, continued Variant 1
// Test stimulus generation
initial
  begin // loop over number of all inputs
        //
    for(i = 0; i < 16; i = i+1) begin
      a_tb <= i;
      for(j = 0; j < 16; j = j+1) begin
        b_tb <= j;
        for (k = 0; k < 2; k = k+1) begin
          Cin <= k;
          #(10);
          expected <= a_tb + b_tb + Cin;
        end
      end
    end
  end
```



Examples from Tim Scherr @ UC

# Adder Test Bench with Self-checking

```
integer i, j, k;
// Adder Testbench, continued Variant 2
// Test stimulus generation
initial
  begin // loop over number of a inputs
        // possible
    for(i = 0; i < 16; i = i+1) begin
      a_tb <= i;
      for(j = 0; j < 16; j = j+1) begin
        b_tb <= j;
        for (k = 0; k < 2; k = k+1) begin
          Cin <= k;
          #(10);
          if (y_tb !== a_tb + b_tb + Cin)
          begin
             $display("Sum is wrong");
             $stop
           end
        end
      end
    end
  end
```

# Adder Test Bench with Assertions – in SystemVerilog

**Mixed Language Support:** Quartus supports mixing Verilog (IEEE 1364) and SystemVerilog (IEEE 1800) modules in the same project

**File Extensions:** .v files → Verilog
.sv files → SystemVerilog

Verilog does not support assertions!
However, SystemVerilog does!

```
assert_label:
  assert (y_tb == a_tb + b_tb + Cin) // SystemVerilog
    begin
      $display("Test passed");
    end
  else
    begin
      $error("Error - sum is wrong");
      $stop;
     end
```



Examples from Tim Scherr @ UC

# Inferred Latches

- Latches = level-sensitive. When the enable (level) is high, they are asynchronous for that duration.
- FPGA engineers almost always avoid using latches in their designs.
- Latches can introduce timing and synchronization issues!
- Their asynchronous nature is less deterministic, making it harder to evaluate the timing performance of a design.
- Often, latches are created inadvertently inferred by incomplete if-else or case statements:

```
module invert (input [1:0] inv, x, output reg o);
    always @(*) begin           // Combinational logic intended
        if (inv == 0) o = x;
        else if(inv == 1) o = ~x;
    end
endmodule
```

← Latch inferred as no value for `o` specified when `inv` = 2 or `inv` = 3.

- In the above, the synthesizer creates a latch for `o`, as it must *remember* the previous value of `o` when `inv` is 2 or 3.

# Inferred Latches

```
module always_combo (
   input A, B, C, D,
   output reg Y);

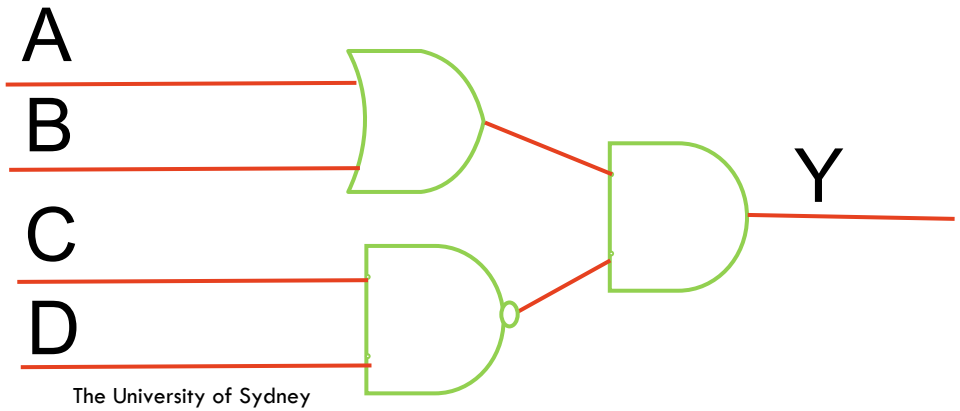
   always @(A or B or C or D)
   begin
     if ((C==1) && (D==1))
       Y <= 0;
     else if ((A==1) || (B==1))
       Y <= 1;
     else
       Y <= 0;
   end
endmodule
```

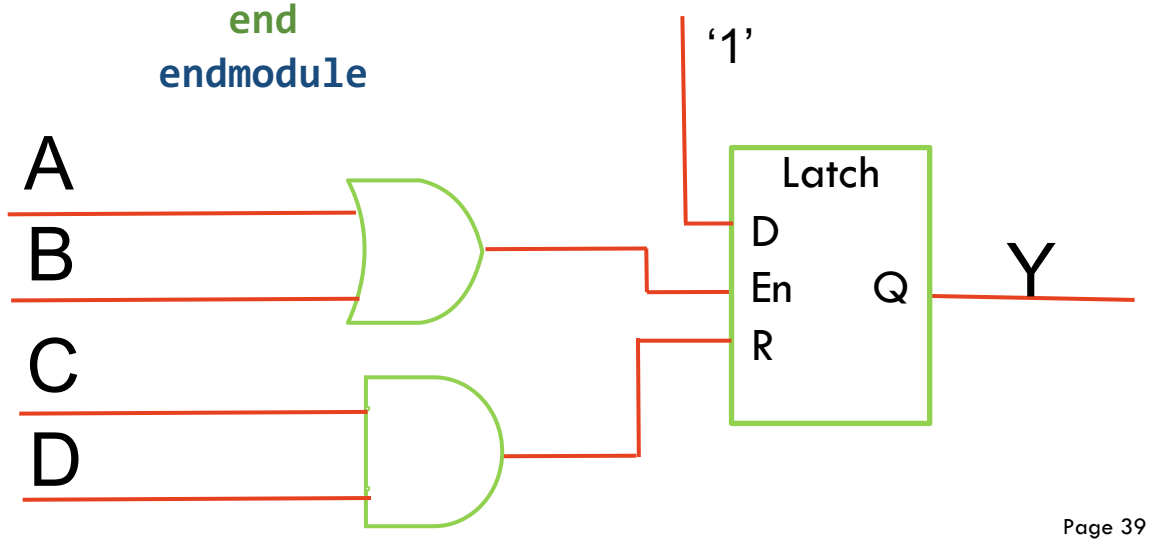
**Vs.**

```
module always_combo (
   input A, B, C, D,
   output reg Y);

   always @(A or B or C or D)
   begin
     if ((C==1) && (D==1))
       Y <= 0;
     else if ((A==1) || (B==1))
       Y <= 1;
     else
       Y <= 0;
   end
endmodule
```

*"Inferred Latch"*



# Inferred Latches

- Techniques to follow to always avoid inferring latches:
  - Before an if-else statement, set the variables assigned in that if-else statement to a default value.
    - Or you can always use an **else** statement to set default values.
  - Always use a **default** case when using a **case** statement and give default values to all variables assigned in the other cases (see EdLesson 2).

```
module invert (input [1:0] inv, x, output reg o);
    always @(*) begin
        o = 0;                  // set `o` to a default value 0.
        if (inv == 0) o = x;
        else if(inv == 1) o = ~x;
    end
endmodule
```

  - In the code above, it is likely that the designer intended `inv` to be a 1-bit input.

# Inferred Latches

Here are the main scenarios where **inferred latches** occur:

**1. Incomplete if statements (most common)**

```
always @(*) begin
    if (sel) y = a;
    // Missing else - y retains old value when sel=0
end
```

**2. Incomplete case statements**

```
always @(*) begin
    case (sel)
        2'b00: y = a;
        2'b01: y = b;
        // Missing 2'b10, 2'b11 cases - y latches
endcase end
```

3. **Variables not assigned in all paths**

```
always @(*) begin
    if (mode == 0) begin
        y = a;
        z = b;
    end
    else begin
        y = c; // z not assigned - z latches!
    end
end
```

4. **Using variables before assignment**

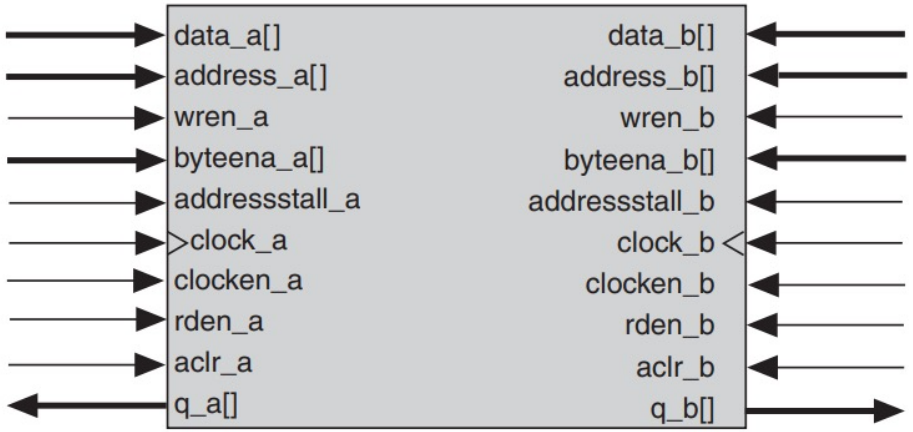
# Hard IP

There are other types of blocks that can be routed with LEs:

- **Hard IP blocks** are predefined circuits implemented directly using dedicated hardware, rather than LUTs.
- Hard IP delivers ASIC-level performance, area efficiency and power efficiency superior to LUT logic, at the expense of being less customizable.
- Hard IP blocks are vendor-specific (i.e. Intel) proprietary Intellectual Property.
- Vendors provide HDL wrapper modules that encapsulate these Hard IP blocks, making them easy for designers to instantiate and use in their FPGA designs. They may also provide simulation code to accurately simulate their IP.

# Block RAM (BRAM)

- Block Random Access Memory (BRAM) is **memory circuitry** designed into the FPGA fabric to provide efficient, high-speed memory access.
- BRAMs are physically optimized for memory operations, so they are more efficient than if a similar memory structure was implemented purely using programmable logic (LEs).
- **Use case:** creating large, deep memory structures with high memory density.




- BRAM can be configured as single-port or dual-port RAM, a ROM, or even a shift register.

# BRAM - Cyclone IV (EP4CE115) vs Cyclone V SoC (5CSEMA5F31C6)

| Feature | Cyclone IV EP4CE115 | Cyclone V SoC 5CSEMA5F31C6 |
|---|---|---|
| **Block Type** | M9K | M10K |
| **Block Size** | 9 Kbits | 10 Kbits |
| **Total Capacity** | 3.888 Mbits | ~4.1 Mbits |
| **Number of Blocks** | 432 blocks | 432 blocks |
| **Maximum Width** | 36 bits | **40 bits** |
| **Maximum Depth** | 1024 words | 1024 words |

The main advantages of the Cyclone V SoC are:

1. **Larger blocks** (10K vs 9K bits)
2. **Wider maximum width** (40 vs 36 bits)
3. **Slightly more total capacity** (~4.1 vs 3.888 Mbits)

Both have the same number of blocks (432), but the newer Cyclone V SoC blocks are more efficient and flexible for memory design.

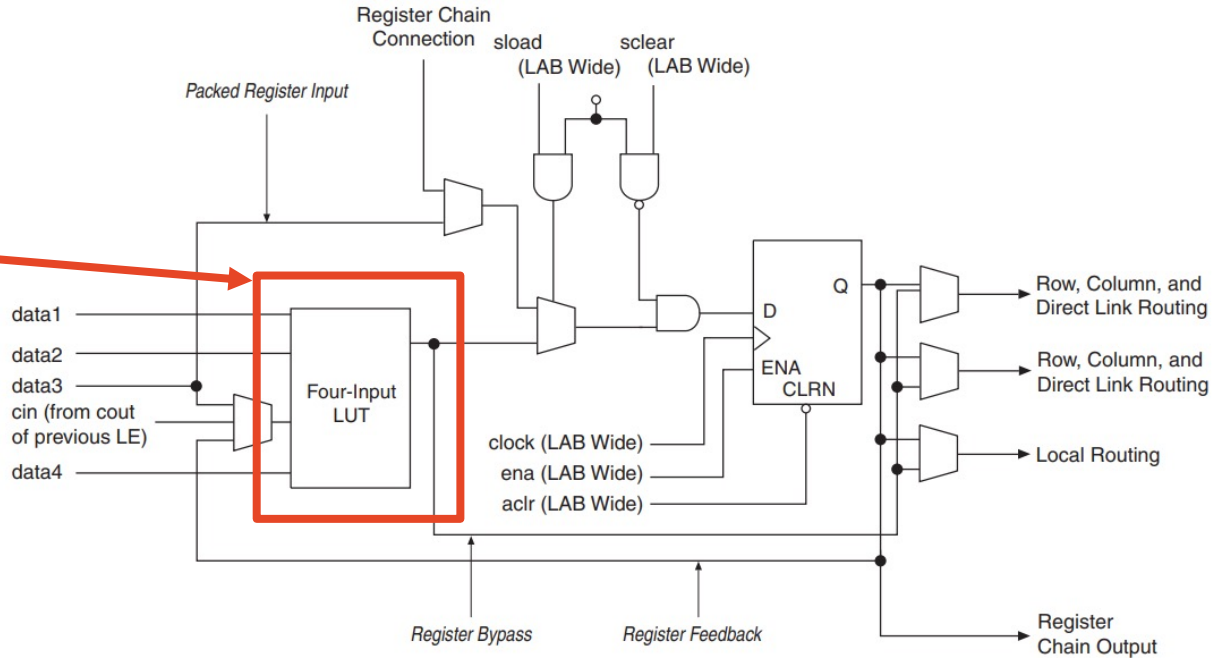
# Distributed RAM

- Distributed RAM is a type of memory that is derived from the FPGA's lookup tables (LUTs).
- An LUT can be configured to function as a small piece of RAM – typically a few bits.
- These small memory units distributed across the FPGA's LE fabric.
- They are perfect for small amounts of memory that is tightly coupled with programmable logic.

**Cyclone IV EP4CE115 LE:**

A 4-input LUT can act as a 1-bit wide memory of 16 elements deep ($2^4$).

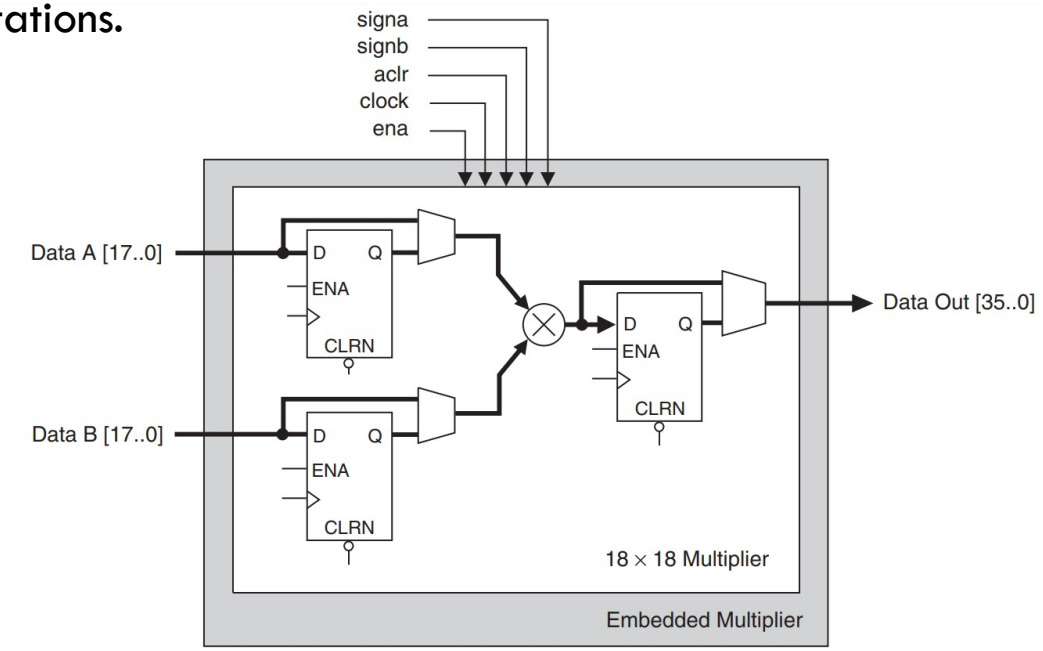
Can achieve larger width with parallel LUTs.



# DSP Blocks (Embedded Multipliers)

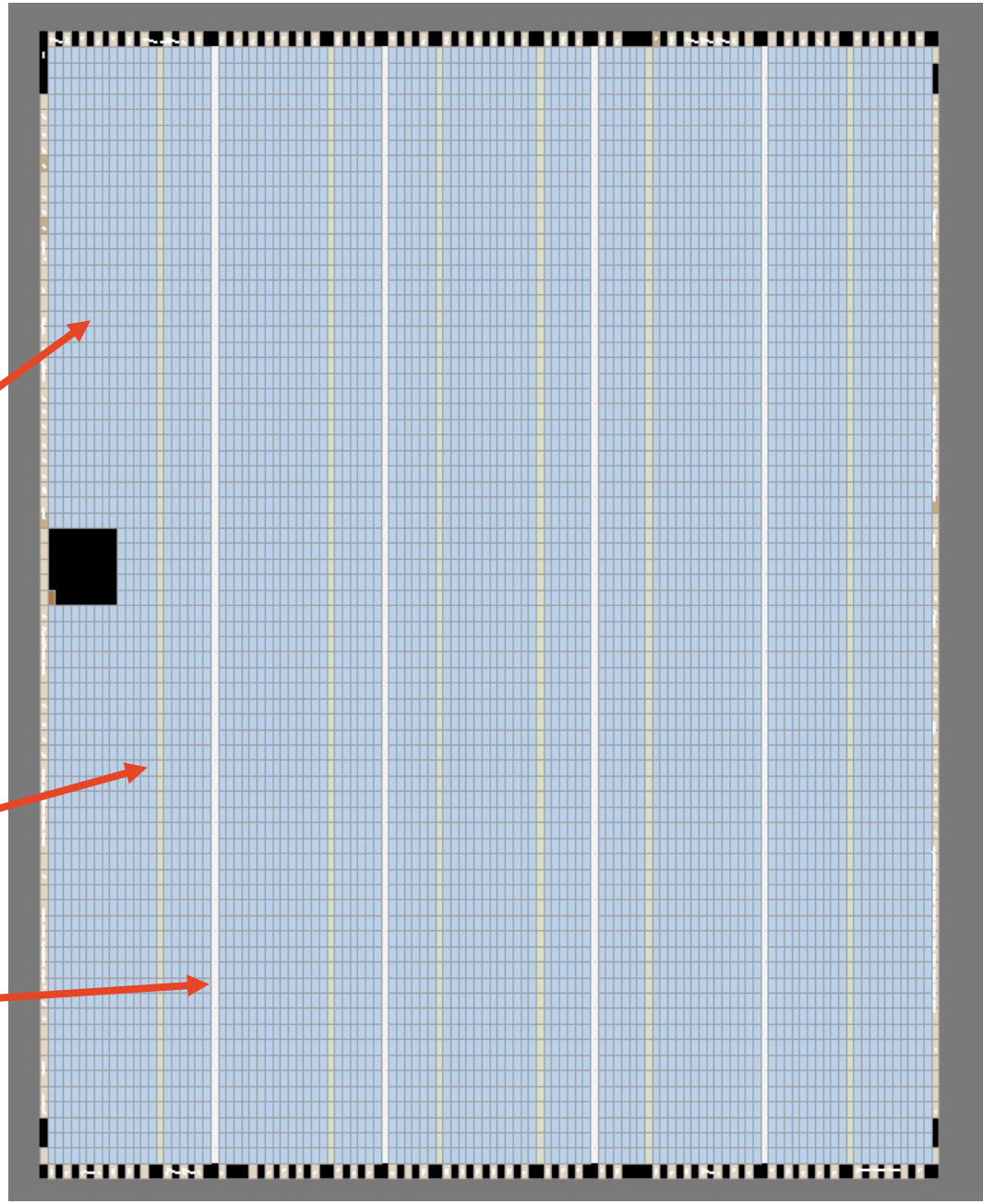
- Digital Signal Processing (DSP) blocks efficiently implement the mathematical operations common in digital signal processing applications: addition, subtraction and multiplication.
- DSP blocks are configurable and can be combined and cascaded to perform more complex operations.
- **Use case:** any applications requiring intensive mathematical computations.

- The Cyclone IV EP4CE115 FPGA has **266 DSP** blocks that *support multiply only* for up to 18-bit inputs. These can also be configured to give 532 9-bit multipliers.
- The Cyclone V FPGA on the DE1-SoC has **112 DSP** blocks that support multiply for up to 27-bit inputs. These can also be configured to give 224 18-bit multipliers or 336 9-bit multipliers.



# Chip Planner

- The Quartus "Chip Planner" tool displays a floormap of the FPGA and visualizes what blocks your design utilizes and how they are routed.
- This is useful for investigating routing congestion.
- BRAM and DSP blocks are arranged in columns.




The Cyclone IV EP4CE115 in the Chip Planner.

# Inferring Block RAM in Verilog

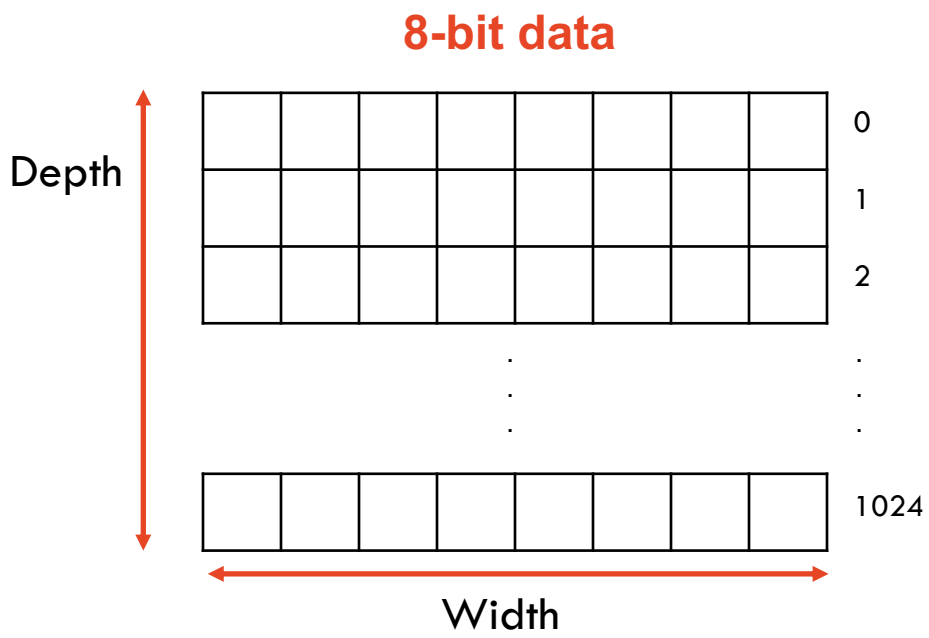
- Now that we have learnt about Block RAM, how do we actually use it?
- Storage elements can be modeled in Verilog using one-dimensional arrays of type reg.
- Each element in the array represents a word and is referred to using a single array index:

```
reg [7:0] my_array [0:255]; // An array of 256 8-bit registers.
                            // my_array[1][0] is bit-0 of the 2nd word in the memory.
```

- Synthesis tools can then infer Block RAM provided that the array is read from and written to *correctly* in a sequential logic **always** block. We will see examples of this in the following slides.
  - See https://www.intel.com/content/www/us/en/docs/programmable/683082/23-1/inferring-memory-functions-from-hdl-code.html.

# Simple Dual Port RAM

```
module DPRAM
#(
  parameter Data_width = 8,  //# of bits in word
            Addr_width = 10  // # of address bits
  )
  (  // ports
    input wire clk,
    input wire we, // Write Enable
    input wire [(Addr_width-1):0] w_addr, r_addr,  // Addresses
    input wire [(Data_width-1):0] d,   // Write Data
    output wire [(Data_width-1):0] q   // Read Data
  );
```



# Simple Dual Port RAM

```
// signal declarations
  reg [Data_width-1:0] ram [2**Addr_width-1:0]; // Array for RAM storage
  reg [Data_width-1:0] data_reg;                // Read output reg.

// RAM initialization from an external file
initial
  $readmemh("initalRAM.txt", ram);
```

# Simple Dual Port RAM

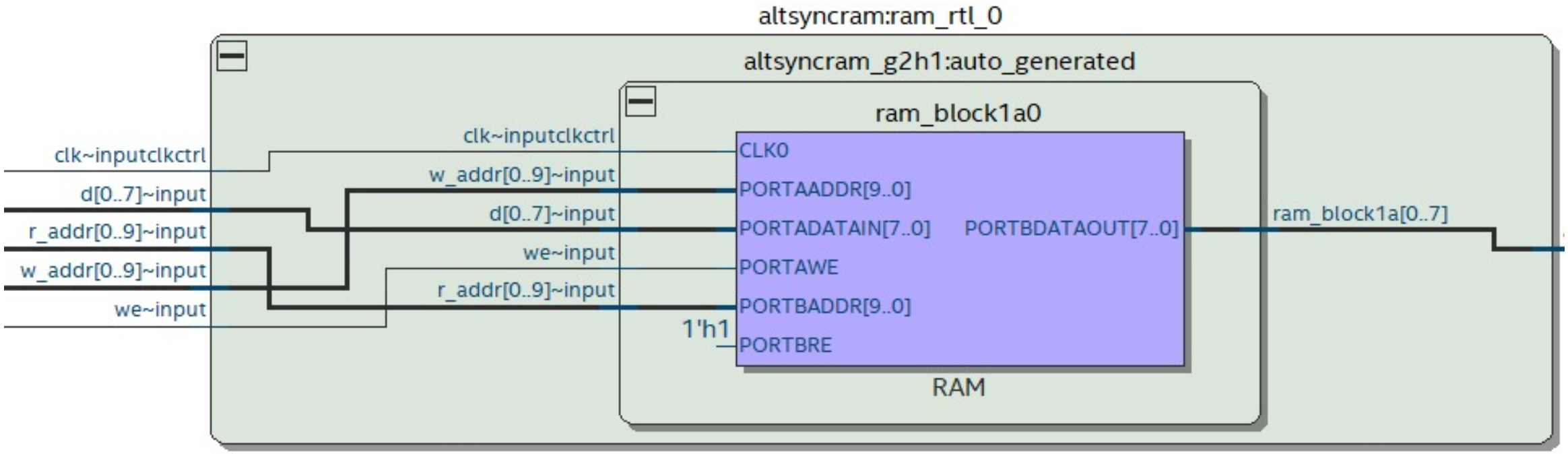
```
// Write operation
  always @(posedge clk) // Read/write from ram (must be sequential logic)
  begin
    if (we) ram[w_addr] <= d;    // write to ram
    data_reg <= ram[r_addr];     // read from ram
  end

// Assign output
  assign q = data_reg;

endmodule
```

# Single Port RAM

- **Result:** You can use the "Technology Map Viewer" to check if the BRAM was inferred correctly, by looking for the `altsyncram` component:



# ROM Example

```
module ROM
#(
  parameter Data_width = 8,  // # of bits in word
            Addr_width = 3   // # of address bits
  )
  (  // ports
    input wire clk,
    input wire [Addr_width-1:0] addr,
    output wire [Data_width-1:0] data
  );

// signal declarations
reg [Data_width-1:0] rom_data, data_reg;
```

# ROM Example

```
always @(posedge clk)
  data_reg <= rom_data; // Read from ROM (must be sequential logic).

always @*
  case(addr)  // Lookup table to initialize ROM.
    3'b000:  rom_data = 8'b1000_0000;
    3'b001:  rom_data = 8'b1010_1010;
    3'b010:  rom_data = 8'b0101_0101;
    3'b011:  rom_data = 8'b1000_0011;
    3'b100:  rom_data = 8'b0000_0000;
    3'b101:  rom_data = 8'b1001_1001;
    3'b110:  rom_data = 8'b1000_0001;
    3'b111:  rom_data = 8'b1111_0000;
  endcase

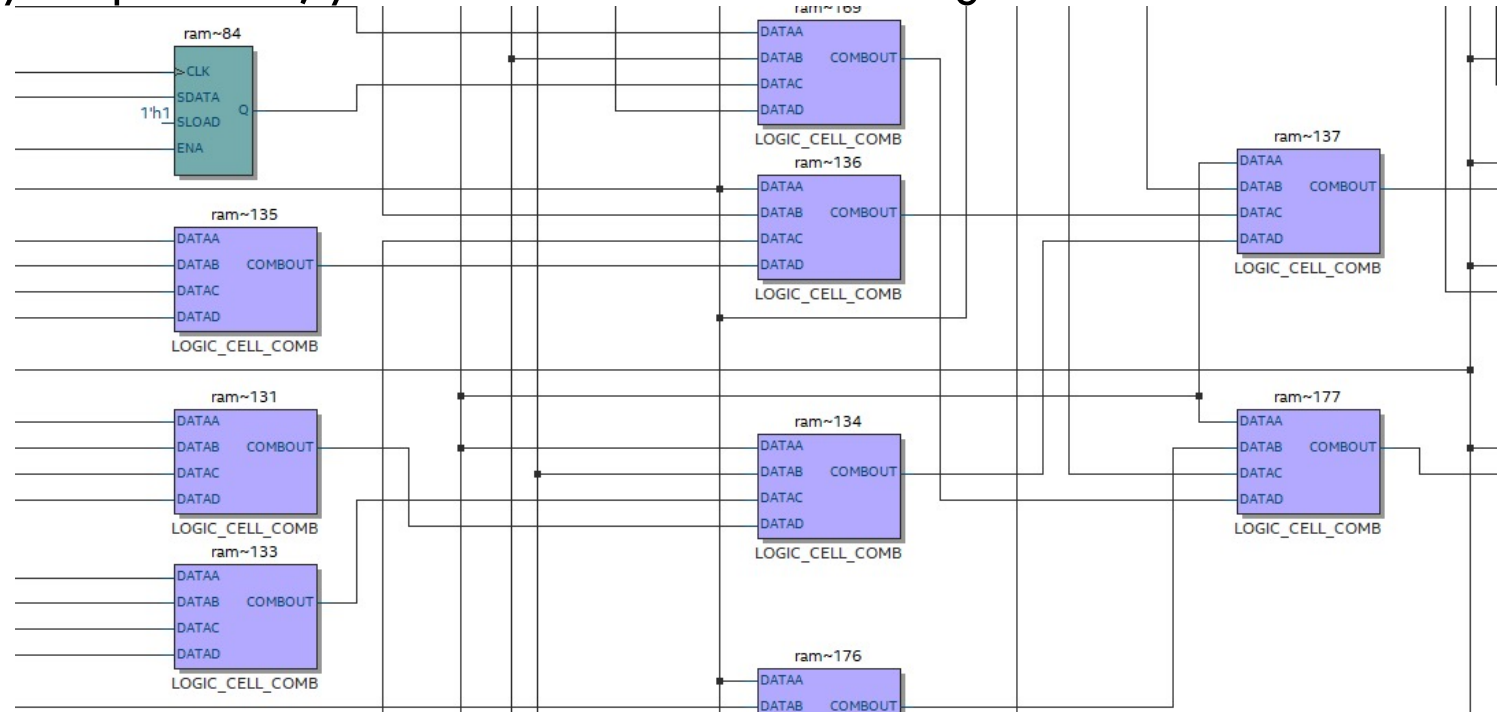
  assign data = data_reg;
endmodule
```

# Inferring Distributed RAM in Verilog

- Quartus will infer Distributed RAM or Block RAM based on the memory depth.
- To force Quartus to use Distributed RAM, you can use the `ramstyle` synthesis directive (Quartus only):

```
(* ramstyle = "logic" *)  reg [Data_width-1:0] ram [2**Addr_width-1:0];
```

- See https://www.intel.com/content/www/us/en/programmable/quartushelp/17.0/hdl/vlog/vlog_file_dir_ram.htm.
- In the Technology Map Viewer, you should see lots of 'ram' logic cells:



# Inferring Multipliers in Verilog

- To infer a multiplier, simply use the multiplication arithmetic operator!

```
module unsigned_mult (out, a, b);
  output [15:0] out;                    (* multstyle = "dsp" *)
  input [7:0] a;
  input [7:0] b;
  assign out = a * b;  // Simply use *
endmodule
```

- See https://www.intel.com/content/www/us/en/docs/programmable/683082/23-1/inferring-multipliers.html
- In the Technology Map Viewer:

